

ADVANCE PROBLEM PROGRAMMING

SLO 1: OOP Principles

1. Write a Class Demonstrating Encapsulation and Inheritance

```
class Person {  
    private String name;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}  
  
class Student extends Person {  
    private int rollNo;  
  
    public void setRollNo(int rollNo) {  
        this.rollNo = rollNo;  
    }  
  
    public void display() {  
        System.out.println("Name: " + getName());  
        System.out.println("Roll No: " + rollNo);  
    }  
}
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        s.setName("Rahul");  
        s.setRollNo(101);  
  
        s.display();  
    }  
}
```

Output:

Name: Rahul

Roll No: 101

Here, name and rollNo are private, demonstrating **encapsulation**. The Student class inherits from the Person class using extends, demonstrating **inheritance**.

2. Implement Abstraction Using an Abstract Class

An abstract class can contain abstract methods whose implementation is provided by its child classes.

```
abstract class Animal {  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}
```

```
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
  
        d.sound();  
        d.eat();  
    }  
}
```

Output:

Dog barks

Animal eats food

Here, Animal is an abstract class and sound() is an abstract method. The Dog class provides the implementation of sound(). This demonstrates **abstraction**.

3. Show Polymorphism with Method Overriding in Java

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal a;  
  
        a = new Dog();  
        a.sound();  
    }  
}
```

```
        a = new Cat();  
        a.sound();  
    }  
}
```

Output:

Dog barks

Cat meows